Universidad Nacional
Vicerrectoría de Docencia
Programa Éxito Académico

Ejercicios: matrices de objetos, herencia y downcasting

Práctica de la semana 14

EIF201 Programación I · II Ciclo 2026 · Tutoría del lunes 19 de octubre

Andrey Jesús Solano Rojas, tutor

Instrucciones

- La **parte A** se resuelve sin computadora: lea el código, prediga el resultado y solo después compruébelo compilando.
- La **parte B** es un programa completo por pasos. Use el archivo `plantilla_bodega.cpp` y resuelva los ejercicios en orden: cada uno se apoya en el anterior.
- La **parte C** son retos sobre la clase `SalaCine` de la guía teórica (archivo `sala_cine.cpp`).
- Antes de programar algo con matrices, dibuje la matriz con sus índices. Antes de liberar memoria, dibuje los tres niveles: objetos, filas y arreglo de filas.

Parte A · Trazado

A1. Recorrido por columnas

```
    int m[3][4] = {{1, 2, 3, 4},
                   {5, 6, 7, 8},
                   {9, 10, 11, 12}};
    for (int c = 0; c < 4; c++)
        for (int f = 0; f < 3; f++)
            cout << m[f][c] << " ";
```

Salida: _____

A2. Recorrido en zigzag

```
    int m[3][4] = {{1, 2, 3, 4},
                   {5, 6, 7, 8},
                   {9, 10, 11, 12}};
    for (int f = 0; f < 3; f++) {
        if (f % 2 == 0)
            for (int c = 0; c < 4; c++)
                cout << m[f][c] << " ";
        else
            for (int c = 3; c >= 0; c--)
                cout << m[f][c] << " ";
    }
```

Salida: _____

A3. Diagonales

```
    int m[4][4] = {{1, 2, 3, 4},
                   {5, 6, 7, 8},
                   {9, 10, 11, 12},
                   {13, 14, 15, 16}};
    int principal = 0, secundaria = 0;
    for (int i = 0; i < 4; i++) {
        principal += m[i][i];
        secundaria += m[i][3 - i];
    }
    cout << principal << " " << secundaria;
```

Salida: _____

A4. ¿Qué parte de la matriz se suma?

Con la misma matriz de 4×4 del ejercicio A3:

```
    int m[4][4] = {{1, 2, 3, 4},
                   {5, 6, 7, 8},
                   {9, 10, 11, 12},
                   {13, 14, 15, 16}};
    int suma = 0;
    for (int f = 0; f < 4; f++)
        for (int c = f + 1; c < 4; c++)
            suma += m[f][c];
    cout << suma;
```

Salida: _____

¿Qué región de la matriz suma este código? _____

A5. ¿Válido o `nullptr`?

Con la jerarquía `Reserva` de la guía teórica:

```
Reserva* r1 = new ReservaVIP("1-1111-1111", "Ana", false);
Reserva* r2 = new ReservaGeneral("2-2222-2222", "Luis");
Reserva* r3 = new ReservaEstudiante("3-3333-3333", "Sofia", "B12345");
Reserva* r4 = nullptr;
```

Indique el resultado de cada línea: *puntero válido*, *nullptr*, *no compila* o *lanza una excepción*.

	Expresión	Resultado
1	`dynamic_cast<ReservaVIP*>(r1)`	
2	`dynamic_cast<ReservaVIP*>(r2)`	
3	`dynamic_cast<ReservaEstudiante*>(r3)`	
4	`dynamic_cast<ReservaEstudiante*>(r1)`	
5	`dynamic_cast<ReservaVIP*>(r4)`	
6	`r1->tieneCombo()`	
7	`dynamic_cast<ReservaVIP&>(*r2)`	

A6. Encuentre los errores

El siguiente constructor y destructor compilan, pero tienen cuatro errores. Identifíquelos y corríjalos.

```
1  SalaCine::SalaCine(int filas, int columnas) : filas(filas), columnas(columnas) {
2      asientos = new Reserva**[filas];
3      for (int f = 0; f < filas; f++)
4          asientos[f] = new Reserva*[columnas];
5  }
6
7  SalaCine::~SalaCine() {
8      delete[] asientos;
9      for (int f = 0; f < filas; f++)
10         delete asientos[f];
11 }
```

A7. Otro error

¿Qué problemas tiene esta versión de `cancelar`? Escriba un caso concreto en el que falle.

```
bool SalaCine::cancelar(int f, int c) {
    delete asientos[f][c];
    return true;
}
```

Parte B · Bodega con estantes

Una bodega organiza sus productos en una matriz: cada fila es un estante y cada columna un espacio del estante. Un espacio está vacío (`nullptr`) o apunta a un producto, y **la bodega es responsable de liberar sus productos**. Hay dos tipos de producto:

Clase	Valor de inventario
`ProductoPerecedero`	Cantidad × precio. Si vence en 3 días o menos, la mitad. Si ya venció (días ≤ 0), cero.
`ProductoElectronico`	Cantidad × precio.

Las clases `Producto`, `ProductoPerecedero` y `ProductoElectronico` ya están implementadas en la plantilla, junto con `Bodega::mapa()` y un `main` de prueba. Esta es la clase que debe completar:

```
class Bodega {
private:
    Producto*** estantes;        // estantes[fila][columna]
    int filas;                   // cantidad de estantes
    int columnas;                // espacios por estante
public:
    Bodega(int filas, int columnas);
    ~Bodega();
    bool posicionValida(int f, int c) const;
    bool ubicar(int f, int c, Producto* p);
    bool retirar(int f, int c);
    bool buscarPorCodigo(string codigo, int& f, int& c) const;
    double valorTotal() const;
    int estanteMasLleno() const;
    bool mover(int f1, int c1, int f2, int c2);
    int contarPorVencer(int dias) const;
    int retirarVencidos();
    string mapa() const;
};
```

B1. Constructor y destructor

- El constructor crea el arreglo de estantes, cada estante y deja todos los espacios en `nullptr`.
- El destructor libera, en orden, los productos, cada estante y el arreglo de estantes.
- Compruebe: con solo B1 resuelto, el programa ya debe mostrar el mapa vacío sin caerse.

B2. Validar, ubicar y retirar

- `posicionValida(f, c)`: `true` si la fila y la columna están dentro de los límites.
- `ubicar(f, c, p)`: guarda el producto si la posición es válida y está libre. Si retorna `false`, el producto no quedó en la bodega y quien llamó debe liberarlo.
- `retirar(f, c)`: libera el producto y deja el espacio en `nullptr`. `false` si la posición no es válida o está vacía.

B3. Buscar y entregar la posición

- `buscarPorCodigo(codigo, f, c)`: si encuentra el producto, deja su posición en los parámetros por referencia `f` y `c` y retorna `true`. Si no, retorna `false`.

B4. Recorridos

- `valorTotal()`: suma el valor de inventario de todos los productos. No debe preguntar de qué tipo es cada producto.
- `estanteMasLleno()`: el estante (fila) con más espacios ocupados; `-1` si la bodega está vacía.

B5. Mover sin copiar

- `mover(f1, c1, f2, c2)`: pasa el producto de un espacio a otro. Ambas posiciones deben ser válidas, el origen ocupado y el destino libre. No debe crear ni liberar ningún objeto.

B6. Downcasting

- `contarPorVencer(dias)`: cantidad de productos perecederos **no vencidos** que vencen en `dias` días o menos.
- `retirarVencidos()`: libera todos los perecederos vencidos, deja sus espacios libres y retorna cuántos retiró.
- Ambos necesitan métodos que solo existen en `ProductoPerecedero` (`getDiasParaVencer()` y `estaVencido()`). Use `dynamic_cast` y verifique el resultado.

Salida esperada

Con los seis ejercicios resueltos, el `main` de la plantilla debe mostrar exactamente:

```
Estante 0: P P . P
Estante 1: E . E .
Estante 2: . . . P
Ubicar en (0,0) ocupado: false
Ubicar en (5,1) invalido: false
Valor total: 136750
Estante mas lleno: 0
Buscar E01: true en (1,2)
Buscar X99: false
Por vencer en 3 dias o menos: 2
Vencidos retirados: 1
Mover (1,2) a (2,0): true
Mover (2,0) a (0,0): false
Retirar (1,0): true
Retirar (1,0) otra vez: false
Estante 0: P P . P
Estante 1: . . . .
Estante 2: E . . .
Valor total: 91750
```

Parte C · Retos sobre la sala de cine

Agregue estos métodos a la clase `SalaCine` de `sala_cine.cpp`. Los ejemplos usan esta sala de 3 × 6:

```
Fila 0: . G . . G .
Fila 1: . V G . . .
Fila 2: . . E . . .
```

C1. Asientos juntos

`int buscarConsecutivos(int f, int n) const`: retorna la columna donde empiezan los primeros `n` asientos libres consecutivos de la fila `f`, o `-1` si no hay. Ejemplos: `buscarConsecutivos(0, 2)` retorna 2; `buscarConsecutivos(0, 3)` retorna -1; `buscarConsecutivos(1, 3)` retorna 3.

C2. Vecinos ocupados

`int vecinosOcupados(int f, int c) const`: cuántos de los hasta ocho asientos que rodean a `(f, c)` están ocupados. Cuidado con los bordes de la matriz. Ejemplos: `vecinosOcupados(1, 2)` retorna 3; `vecinosOcupados(0, 0)` retorna 2.

C3. Agregar una fila

`void agregarFila()`: agrega una fila vacía al final de la sala, sin perder ninguna reserva. Pista: hay que crear un arreglo de filas más grande, pero las filas existentes *no* se copian ni se liberan.

C4. Un tipo nuevo de reserva

Agregue la clase `ReservaAdultoMayor`, con un precio de 1750 colones y el símbolo `A`. ¿Qué métodos de `SalaCine` tuvo que modificar para que `recaudacion()` y `mapa()` la tomen en cuenta? ¿Por qué?